DBMS - SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: Duke Bike - Sales and Customer Analytics System

Objectives

- Apply COUNT(), SUM(), AVG(), MIN(), and MAX().

- Generate total sales reports.

- Find top customers based on purchase amount.

- Identify best-selling products.

- Perform category-wise sales analysis.

Database Tables Used

The Duke Bike analytics queries use the existing Customer, Product, Orders, and Order_Details tables.

Table	Columns used in the analytics
Customer	ID, NAME
Product	ProductID, ProductName, Category
Orders	Order_ID, Customer_ID, Order_Date, Total_Amount
Order_Details	Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal

Data Key Alignment

The first JOIN attempts returned no rows because the customer IDs in Orders (101-105) and product IDs in Order_Details (201-205) did not match the corresponding IDs in Customer (1-5) and Product (101-106). The following updates were executed and committed before the successful final product and combined reports:

```
UPDATE Orders
SET Customer_ID = Customer_ID - 100
WHERE Customer_ID BETWEEN 101 AND 105;

UPDATE Order_Details
SET Product_ID = Product_ID - 100
WHERE Product_ID BETWEEN 201 AND 205;

COMMIT;
```

The report uses the successful outputs supplied after the ID alignment. No additional database changes are made by this document.

1. Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a single result. The following queries use the Total_Amount column in Orders.

1.1 COUNT() - Find the total number of orders

```
SELECT COUNT(*) AS Total_Orders
FROM Orders;
```

1.2 SUM() - Find the total sales amount

```
SELECT SUM(Total_Amount) AS Total_Sales
FROM Orders;
```

1.3 AVG() - Find the average order amount

```
SELECT AVG(Total_Amount) AS Average_Order_Amount
FROM Orders;
```

1.4 MIN() - Find the lowest order amount

```
SELECT MIN(Total_Amount) AS Lowest_Order_Amount
FROM Orders;
```

1.5 MAX() - Find the highest order amount

```
SELECT MAX(Total_Amount) AS Highest_Order_Amount
FROM Orders;
```

The supplied SQL output shows 5 orders, total order sales of 507,200, an average order amount of 101,440, a lowest order amount of 4,200, and a highest order amount of 215,000.

```
SQL> SELECT COUNT(*) AS Total_Orders
  2  FROM Orders;

TOTAL_ORDERS
------------
           5

SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
     507200

SQL> SELECT MIN(Total_Amount) AS Lowest_Order_Amount
  2  FROM Orders;

LOWEST_ORDER_AMOUNT
-------------------
               4200

SQL> SELECT MAX(Total_Amount) AS Highest_Order_Amount
  2  FROM Orders;

HIGHEST_ORDER_AMOUNT
--------------------
              215000

SQL> SELECT
  2      COUNT(Order_ID) AS Total_Orders,
  3      SUM(Total_Amount) AS Total_Sales,
  4      AVG(Total_Amount) AS Average_Sales
  5  FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_SALES
```

SQL output: COUNT(), SUM(), MIN(), MAX(), and combined sales summary.

2. Generate Total Sales Reports

A total sales report can be generated using SUM() on the Total_Amount column in Orders.

```
SELECT SUM(Total_Amount) AS Total_Sales
FROM Orders;
```

Total sales from the Orders table: 507,200.

2.1 Customer-wise Sales Report

This report shows the customer ID, customer name, total number of orders, and total purchase amount for each customer.

```
SELECT
    C.ID AS Customer_ID,
    C.NAME AS Customer_Name,
    COUNT(O.Order_ID) AS Total_Orders,
    SUM(O.Total_Amount) AS Total_Purchase_Amount
FROM Customer C
LEFT JOIN Orders O
ON C.ID = O.Customer_ID
GROUP BY C.ID, C.NAME
ORDER BY Total_Purchase_Amount DESC;
```

3. Find Top Customers Based on Purchase Amount

SUM(), GROUP BY, ORDER BY, and ROWNUM are used to display the five customers with the highest purchase amounts in Oracle.

```
SELECT *
FROM (
    SELECT
        C.ID AS Customer_ID,
        C.NAME AS Customer_Name,
        SUM(O.Total_Amount) AS Total_Purchase_Amount
    FROM Customer C
    JOIN Orders O
    ON C.ID = O.Customer_ID
    GROUP BY C.ID, C.NAME
    ORDER BY Total_Purchase_Amount DESC
)
WHERE ROWNUM <= 5;
```

The first attempt at these customer JOIN reports was made before the customer IDs were aligned and returned zero matching orders. The final combined report supplied afterwards confirms the five customer-order matches. The following purchase amounts are read from the Orders records supplied in the screenshots; they are not presented as a new SQL output screenshot:

Customer ID	Customer Name	Orders	Purchase Amount
5	Hardik	1	215,000
1	Ramesh	1	190,000
3	Kaushik	1	85,000
2	Khilan	1	13,000
4	Chaitali	1	4,200

Note: The final customer-wise and top-customer SQL output screenshots after the ID correction were not included in the supplied images. The table above is based on the final customer/order links and Orders amounts visible in the supplied records.

4. Identify Best-Selling Products

Order_Details contains the quantity of each product ordered. SUM(Quantity) is used to find the total quantity sold for each product.

```
SELECT
    P.ProductID,
    P.ProductName,
    SUM(OD.Quantity) AS Total_Quantity_Sold
FROM Product P
JOIN Order_Details OD
ON P.ProductID = OD.Product_ID
GROUP BY P.ProductID, P.ProductName
ORDER BY Total_Quantity_Sold DESC;
```

Purpose: This query connects Product and Order_Details, groups records by product, calculates total quantity sold, and displays products from highest to lowest quantity sold.

```
SQL> SELECT
  2      P.ProductID,
  3      P.ProductName,
  4      SUM(OD.Quantity) AS Total_Quantity_Sold
  5  FROM Product P
  6  JOIN Order_Details OD
  7  ON P.ProductID = OD.Product_ID
  8  GROUP BY P.ProductID, P.ProductName
  9  ORDER BY Total_Quantity_Sold DESC;

 PRODUCTID PRODUCTNAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       102 Duke 250
                 2

       105 Bike Gloves
                 1

       104 Helmet
                 1


 PRODUCTID PRODUCTNAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101 Duke 200
                 1

       103 Duke 390
                 1
```

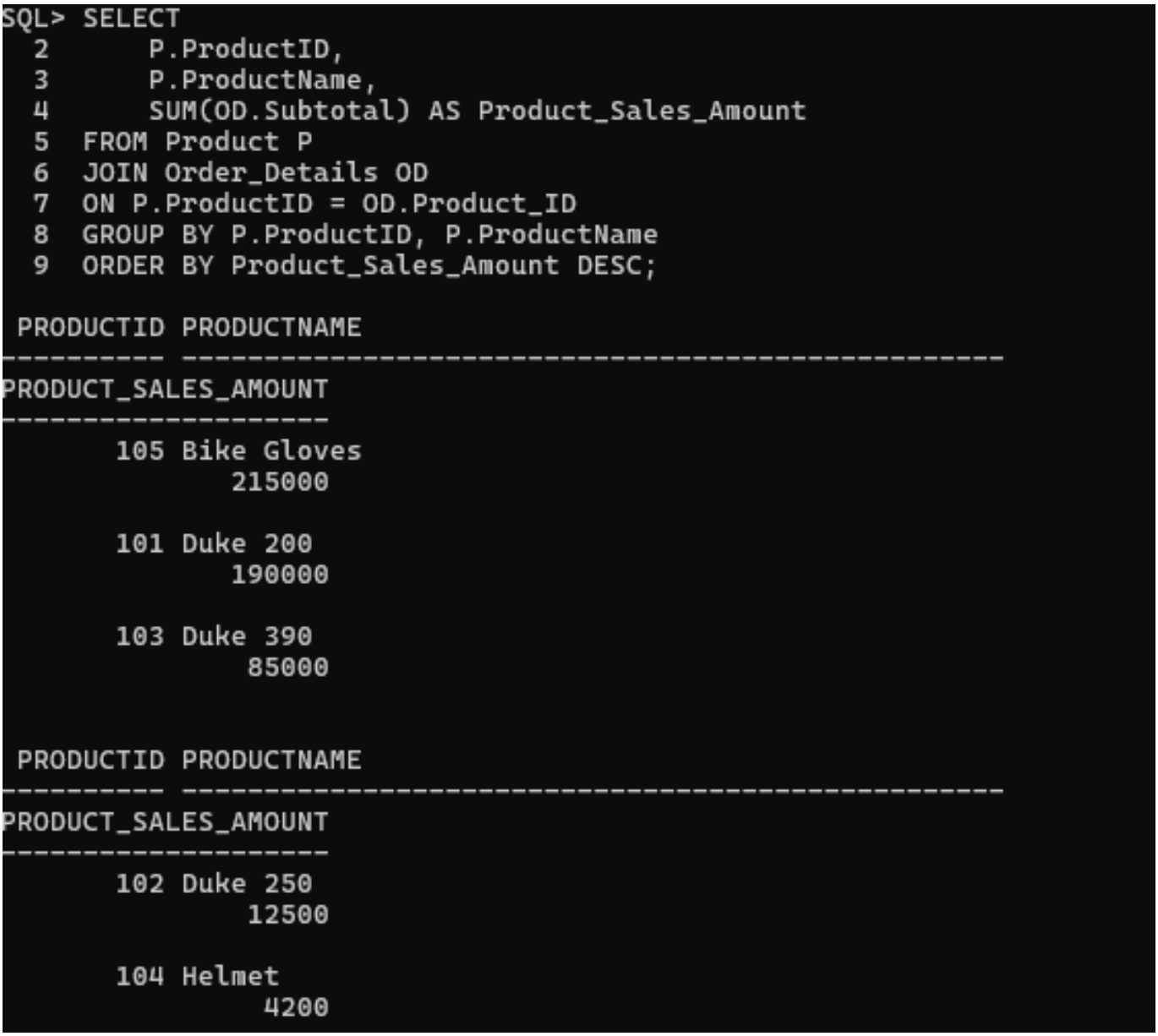
SQL output: total quantity sold for each Duke Bike product.

Duke 250 has the highest quantity sold, with 2 units. Bike Gloves, Helmet, Duke 200, and Duke 390 each have 1 unit sold.

4.1 Product Sales Amount

Product sales amount calculates the revenue generated by each product using the Subtotal column in Order_Details.

```
SELECT
    P.ProductID,
    P.ProductName,
    SUM(OD.Subtotal) AS Product_Sales_Amount
FROM Product P
JOIN Order_Details OD
ON P.ProductID = OD.Product_ID
GROUP BY P.ProductID, P.ProductName
ORDER BY Product_Sales_Amount DESC;
```



SQL output: sales amount generated by each product.

Bike Gloves generated the highest product sales amount (215,000), followed by Duke 200 (190,000), Duke 390 (85,000), Duke 250 (12,500), and Helmet (4,200).

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category.

```
SELECT
    P.Category,
    SUM(OD.Subtotal) AS Category_Sales
FROM Product P
JOIN Order_Details OD
ON P.ProductID = OD.Product_ID
GROUP BY P.Category
ORDER BY Category_Sales DESC;
```

```
SQL> SELECT
  2       P.Category,
  3       SUM(OD.Subtotal) AS Category_Sales
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.ProductID = OD.Product_ID
  7  GROUP BY P.Category
  8  ORDER BY Category_Sales DESC;

CATEGORY                       CATEGORY_SALES
------------------------------ --------------
Bike                                   287500
Accessories                            219200
```

SQL output: sales generated by each product category.

The Bike category generated 287,500 in sales, while Accessories generated 219,200 in sales.

Sales Data Consistency Note

The Orders table total is 507,200, while the product/category totals from Order_Details sum to 506,700. The 500 difference is visible in the supplied data: Order 5002 has Total_Amount = 13,000 in Orders, but its Order_Details Subtotal remains 12,500. The figures are shown as they appeared in the submitted SQL outputs; no further change was made.

6. Combined Business Sales Report

A complete sales report combines customer, order, product, and category information.

```
SELECT
    C.ID AS Customer_ID,
    C.NAME AS Customer_Name,
    O.Order_ID,
    O.Order_Date,
    P.ProductName,
    P.Category,
    OD.Quantity,
    OD.Unit_Price,
    OD.Subtotal AS Amount_Generated
FROM Customer C
JOIN Orders O
ON C.ID = O.Customer_ID
JOIN Order_Details OD
ON O.Order_ID = OD.Order_ID
JOIN Product P
ON OD.Product_ID = P.ProductID
ORDER BY O.Order_Date DESC;
```

Report structure: This query displays who purchased the product, which order was placed, the product name, its category, quantity purchased, unit price, and amount generated.

```
SQL> SELECT
  2      C.ID AS Customer_ID,
  3      C.NAME AS Customer_Name,
  4      O.Order_ID,
  5      O.Order_Date,
  6      P.ProductName,
  7      P.Category,
  8      OD.Quantity,
  9      OD.Unit_Price,
 10      OD.Subtotal AS Amount_Generated
 11  FROM Customer C
 12  JOIN Orders O
 13  ON C.ID = O.Customer_ID
 14  JOIN Order_Details OD
 15  ON O.Order_ID = OD.Order_ID
 16  JOIN Product P
 17  ON OD.Product_ID = P.ProductID
 18  ORDER BY O.Order_Date DESC;

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID ORDER_DAT
----------- -------------------- ---------- ---------
PRODUCTNAME
--------------------------------------------------
CATEGORY                       QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------
          1 Ramesh                     5001 21-SEP-26
Duke 200
Bike                                    1     190000           190000

          2 Khilan                     5002 21-SEP-26
Duke 250
Bike                                    2       6250            12500

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID ORDER_DAT
----------- -------------------- ---------- ---------
PRODUCTNAME
--------------------------------------------------
CATEGORY                       QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------

          3 Kaushik                    5003 20-SEP-26
Duke 390
Bike                                    1      85000            85000
```

SQL output: combined customer, order, product, and category report (first part).

6. Combined Business Sales Report - Output

```
CATEGORY                       QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------
         1 Ramesh                     5001 21-SEP-26
Duke 200
Bike                                     1     190000           190000

         2 Khilan                     5002 21-SEP-26
Duke 250
Bike                                     2       6250            12500

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID ORDER_DAT
----------- -------------------- ---------- ---------
PRODUCTNAME
----------------------------------------------
CATEGORY                       QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------

         3 Kaushik                    5003 20-SEP-26
Duke 390
Bike                                     1      85000            85000

         4 Chaitali                   5004 20-SEP-26
Helmet

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID ORDER_DAT
----------- -------------------- ---------- ---------
PRODUCTNAME
----------------------------------------------
CATEGORY                       QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------
Accessories                              1       4200             4200

         5 Hardik                     5005 19-SEP-26
Bike Gloves
Accessories                              1     215000           215000


SQL>
```

SQL output: continuation of the combined Duke Bike sales report.

Conclusion

The Duke Bike Sales and Customer Analytics System demonstrates the use of SQL aggregate functions, total sales calculations, customer-wise purchase analysis, top-customer ranking, best-selling product identification, product-wise sales amount, category-wise sales analysis, and a combined business report using table joins.

The final combined report contains five customer orders: Ramesh, Khilan, Kaushik, Chaitali, and Hardik. The supplied output screenshots are included as the execution evidence for the corresponding SQL sections.